

MapanSetu

SIH26036 — Proper Execution Plan

Current development position: The major multi-role frontend workflows are implemented. The next milestone is to convert the high-fidelity prototype into a real end-to-end system by implementing the Django/PostgreSQL backend, durable evidence storage, synchronization, cryptographic certificates, testing, and deployment.

1. Current Status

Area	Current Status	Approx.
Frontend UI / UX	Very strong; major screens/workflows implemented	90–95%
Authentication UI + RBAC UI	Implemented; backend enforcement pending	90%
Business portal	UI/prototype complete	90%
Admin portal	UI/prototype complete	90%
Field Officer portal	UI/prototype complete	90%
Offline workflow	Functional prototype; IndexedDB migration pending	60%
Django backend	Major implementation remaining	10%
PostgreSQL database	Models/migrations largely remaining	0–10%
Real API integration	Major implementation remaining	20%
Certificate engine	Frontend simulation; server engine pending	30–35%
Audit system	Frontend simulation; backend persistence pending	30–35%
Testing / QA	Build/lint verified; automated suites pending	15%
Deployment	Configuration/containerization pending	20%

2. What Is Already Built

- **Authentication & RBAC:** Login experience, role-aware guards, guest protection, session handling, and sign-out flows.
- **Public verification:** Public certificate route, status rendering, minimal disclosure, copy/print interactions.
- **Business workspace:** Dashboard, instrument registry, instrument registration, passport, applications, tracking, certificates, notifications, and profile.
- **Admin workspace:** Dashboard, application triage, officer assignment, scheduling, certificate revocation, officer directory, schedules, audit trail, registry, notifications, and settings.
- **Field workspace:** Assigned cases, checklist, measurements, evidence, review/decision, sync center, profile/storage manager, and offline simulation.

3. Core Execution Principle

Do not spend the next phase building more frontend screens. The existing screens should now be connected to a real system of record. The priority is to make one complete vertical slice work from Business registration through public certificate verification.

4. Phase-by-Phase Execution Plan

Phase 1 — Database Foundation (P0 · 2–3 days)

- Implement CustomUser, Business, Instrument, Application, Inspection, InspectionReading, InspectionChecklist, Evidence, Certificate, AuditLog, SyncOperation, and Notification models.
- Add enums, foreign keys, indexes, uniqueness constraints, timestamps, and optimistic-locking version fields.
- Generate and apply Django migrations.
- Seed baseline demo accounts/data.

Phase 2 — Authentication + RBAC (P0 · 1–2 days)

- Implement POST /api/v1/auth/login, POST /api/v1/auth/refresh, POST /api/v1/auth/logout, and GET /api/v1/users/me.
- Use SimpleJWT and Argon2id.
- Implement IsAdminRole, IsOfficerRole, and IsBusinessRole.
- Enforce object-level ownership, not only frontend route protection.

Phase 3 — Business + Instruments + Applications (P0 · 3–4 days)

- Implement business profile GET/PUT APIs.
- Implement instrument list/create/retrieve/update/passport APIs.
- Add duplicate instrument protection and safe filters/pagination.
- Implement application create/list/retrieve APIs.
- Enforce canonical application state transitions.

Phase 4 — Admin Assignment + Scheduling (P0 · 2 days)

- Implement officer listing and admin dashboard aggregation.
- Implement application assignment and inspection scheduling endpoints.
- Validate officer/application state and jurisdiction rules where applicable.
- Create audit records for administrative actions.

Phase 5 — Officer Inspection Backend (P0 · 3 days)

- Implement inspection creation/submission.
- Persist checklist, readings, result, remarks, and timestamps.
- Validate $e = I - R$ on the backend.
- Support PASS, FAIL, and REQUIRES_CORRECTION.

Phase 6 — Evidence + MinIO (P0 · 2–3 days)

- Implement presigned upload URL generation and evidence confirmation.
- Store binary evidence in MinIO, not PostgreSQL.
- Persist MIME type, size, SHA-256, GPS, accuracy, category, object key, and capture time.
- Validate uploads server-side.

Phase 7 — Offline Sync + Idempotency (P0 · 3 days)

- Move large offline data from localStorage to IndexedDB/Dexie.
- Store cached cases, inspection drafts, binary evidence, and sync queue.

- Implement POST /api/v1/sync.
- Use UUID clientOperationId as an idempotency key so duplicate submissions are processed once.

Phase 8 — Conflict Resolution (P1 · 1–2 days)

- Use version fields for optimistic locking.
- Detect stale offline updates.
- Return HTTP 409 Conflict when appropriate.
- Provide a clear frontend resolution path.

Phase 9 — Certificate Engine (P0/P1 · 3 days)

- Generate certificate payload from a passed inspection.
- Calculate canonical SHA-256 payload hash.
- Generate RSA-PSS signature using the server private key.
- Generate QR code targeting /verify/{certificateNumber}.
- Generate official PDF with ReportLab and store it in MinIO.

Phase 10 — Public Verification (P0 · 1 day)

- Implement public GET /api/v1/certificates/verify?certNo=....
- No authentication required.
- Return only the minimum public verification information.
- Connect the existing /verify/[certNo] page to the real endpoint.

Phase 11 — Audit Chain (P1 · 2 days)

- Persist security-sensitive state changes in AuditLog.
- Create sequential SHA-256 hash chaining using previousHash/currentHash.
- Expose the audit API to the existing admin audit screen.

Phase 12 — Notifications (P1 · 1 day)

- Implement notification list and mark-as-read APIs.
- Generate lifecycle notifications for submission, assignment, scheduling, completion, issuance, and revocation.

Phase 13 — Frontend API Switchover (P0 · 2 days)

- Connect frontend services to live Django endpoints.
- Remove demo/localStorage fallbacks where real APIs are ready.
- Retain deliberate offline behavior for field workflows.
- Verify all role-specific screens against live data.

Phase 14 — Testing + Security (P1 · 3–4 days)

- Write Django unit/model tests.
- Write RBAC, ownership, rate-limit, sync-idempotency, conflict, and cryptography tests.
- Add Playwright multi-persona E2E tests.
- Perform OWASP-focused security checks.

Phase 15 — Deployment (P2 · 2–3 days)

- Containerize frontend, backend, PostgreSQL, MinIO, and Redis as required.
- Add health checks and environment validation.
- Configure production secrets and storage.
- Run a clean deployment and smoke test.

5. Recommended Development Order

1. Database Models & Migrations
2. Authentication + RBAC
3. Business / Instrument / Application APIs
4. Admin Assignment + Scheduling
5. Officer Inspection API
6. MinIO Evidence
7. Offline Sync + Idempotency
8. Conflict Resolution
9. Certificate Generation + Signing
10. Public Verification
11. Audit Chain
12. Notifications
13. Frontend API Switchover
14. Testing + Security
15. Docker + Deployment

6. Requirement Completion View

Category	Current position	What remains
FR-001–FR-018	Mostly frontend / partial	Backend auth, ownership, domain APIs, assignment, scheduling, inspection models
FR-019–FR-028	Mixed / prototype	Server validation, evidence storage, IndexedDB, sync endpoint, conflict handling
FR-029–FR-033	Frontend simulation	Certificate generation, signing, PDF, QR, verification API, revocation backend
FR-034–FR-040	Mostly UI/prototype	Notifications, filters, dashboard aggregation, audit persistence, officer APIs
FR-041	Complete	Maintain demo disclaimers
FR-042–FR-043	Optional / partial	AI advisory can remain lower priority and must remain non-autonomous

7. Critical MVP Vertical Slice

The highest-value milestone is to prove one complete real workflow rather than finishing isolated screens:

- ✓ Business logs in with a real JWT.
- ✓ Business registers an instrument in PostgreSQL.
- ✓ Business submits an application.
- ✓ Admin views it and assigns an officer.
- ✓ Admin schedules the inspection.
- ✓ Officer receives/caches the assigned case.
- ✓ Officer completes checklist and measurements.
- ✓ Officer captures evidence and can work offline.
- ✓ Officer syncs using clientOperationId.

- ✓ Backend validates the inspection and records the result.
- ✓ A PASS result triggers certificate generation.
- ✓ Certificate receives SHA-256 hash, RSA-PSS signature, PDF, and QR code.
- ✓ Public user verifies the certificate without logging in.

8. Definition of 'Project Ready'

- All core records persist in PostgreSQL.
- No role can access another role's or business's unauthorized data.
- The complete Business → Admin → Officer → Certificate → Public Verification flow works against live APIs.
- Offline inspection drafts survive reload and can sync safely.
- Duplicate sync requests do not duplicate database operations.
- Evidence is stored durably in MinIO.
- Certificates are generated and cryptographically signed on the server.
- Audit events are persisted and hash-linked.
- Automated backend/API/E2E tests cover the critical workflow.
- The application can be deployed reproducibly using the project deployment configuration.

9. Immediate Next Actions

1. Create the PR for the current frontend milestone.
2. Start Phase 1: implement Django models and migrations.
3. Implement real authentication immediately after the database foundation.
4. Build the Business → Application → Admin → Officer vertical slice before adding secondary features.
5. Remove demo fallbacks only as their corresponding live APIs become stable.

10. Final Project Position

Frontend / user-facing implementation: approximately 90–95% of the planned interface and workflow surface is built.

Production-grade system implementation: approximately 35–45%, because substantial backend, persistence, storage, cryptographic, integration, testing, and deployment work remains.

Recommended project description: “The complete multi-role frontend workflow and offline inspection prototype are implemented. The current development milestone is integrating these workflows with the Django/PostgreSQL backend, durable evidence storage, cryptographic certificate engine, and automated testing.”